

ZADDP

Zombie API Discovery & Defence Platform

Product Requirements Document

Discovery Scanner Module

v1.0 — Internal Draft

Document:	Product Requirements Document — Discovery Scanner Module
Project:	ZADDP — Zombie API Discovery & Defence Platform
Version:	1.0
Status:	Draft
Author:	ZADDP Engineering Team
Date:	May 2026

Table of Contents

- 1. Overview
 - 2. Goals & Success Metrics
 - 3. Architecture
 - 4. Functional Requirements
 - 5. Technical Design
 - 6. Non-Functional Requirements
 - 7. Implementation Phases & Effort
 - 8. Edge Cases
 - 9. Open Questions
-

1. Overview

The **Discovery Scanner** is a built-in ZADDP module that uses a headless browser (Playwright) to authenticate to target websites via OAuth/SSO, crawl all pages, intercept API calls, discover endpoints, classify them against ZADDP’s known inventory, and drive the decommission workflow for shadow and zombie APIs.

Target Users: Security engineers, API platform owners, compliance auditors at financial institutions.

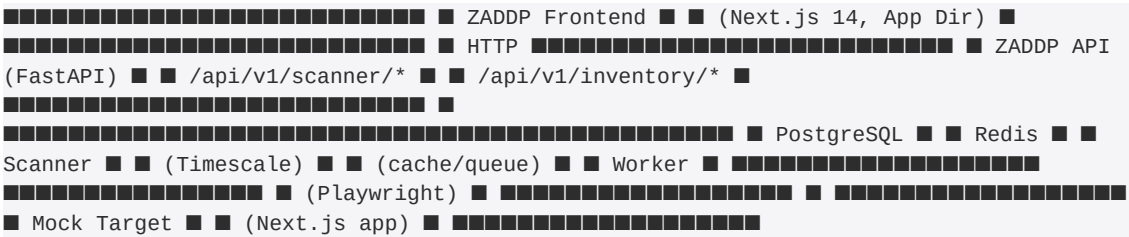
Value Proposition: Automates the manual, error-prone process of API discovery. Replaces periodic spreadsheet audits with continuous, scan-driven inventory validation. Finds shadow APIs (unknown to security teams) and zombie APIs (unused but still live) in minutes.

2. Goals & Success Metrics

Goal	Success Metric
Discover all API endpoints from a target web app	≥95% of live API endpoints captured per scan
Correctly classify discovered APIs	≥90% precision/recall vs manual audit of mock target
Reduce time to decommission zombie APIs	From days/weeks (manual) to <1 hour after scan review
Surface shadow APIs automatically	Zero shadow APIs go undetected after first scan of mock target

3. Architecture

The scanner consists of the following components:



- Scanner Worker** — Separate Docker container running a Python service with Playwright + Chromium. Listens for scan jobs via Redis queue.
- ZADDP API** — FastAPI backend with new /api/v1/scanner/* endpoints. Enqueues scan jobs to Redis, stores results in PostgreSQL.
- Mock Target** — Separate Next.js app simulating a banking website with documented, shadow, deprecated, and zombie APIs.

4. Functional Requirements

FR1: Scan Target Management

ID	Requirement	Priority
FR1.1	User can create a scan target with: name, target URL, OAuth client ID, OAuth client secret, OAuth token URL, OAuth scopes	P0
FR1.2	User can optionally configure cookie-based auth as fallback	P1
FR1.3	User can list, edit, delete scan targets	P0
FR1.4	User can test OAuth connection before running a full scan	P1

FR2: Scan Execution

ID	Requirement	Priority
FR2.1	User triggers a scan on a target — creates a scan_job and queues it to Redis	P0
FR2.2	Scanner Worker picks up the job, launches Playwright headless Chromium	P0
FR2.3	Scanner performs OAuth 2.0 Authorization Code flow to obtain access token	P0
FR2.4	Scanner injects token into browser, navigates to target URL	P0
FR2.5	Scanner crawls all discoverable pages (follows links, clicks buttons) up to configurable depth (default: 3)	P0
FR2.6	Scanner intercepts all network requests via page.route() — captures method, URL, status, bodies, content-type	P0
FR2.7	After crawl, scanner performs fuzzing phase — tries common API paths (from wordlist) against target domain	P1
FR2.8	Scanner normalizes URLs: replaces UUIDs, numeric IDs, hashes with {param} placeholders	P0
FR2.9	Scanner deduplicates endpoints by (method, normalized_path)	P0
FR2.10	Scan status is updated in real-time via periodic polling (or WebSocket)	P1

FR3: Classification

ID	Requirement	Priority
FR3.1	Scanner compares discovered endpoints against ZADDP's apis table (method + normalized path matching)	P0
FR3.2	Classification labels: Matched (exists in inventory), Shadow (not in inventory), Zombie (in inventory as active but not found), Deprecated Still Active (in inventory as deprecated but still responding)	P0
FR3.3	Results stored in scan_results table linked to the scan_job	P0
FR3.4	Scanner generates summary stats: total discovered, matched, shadow, zombie counts	P0

FR4: Results Review & Actions

ID	Requirement	Priority
FR4.1	User views scan results in a paginated table: endpoint, method, classification, status code, details	P0

FR4.2	User can filter results by classification (matched / shadow / zombie / deprecated)	P0
FR4.3	User can bulk-import shadow APIs into the inventory — creates new API records with status=shadow, auto-populates endpoint_path and http_method	P0
FR4.4	User can bulk-decommission zombie APIs — creates decommission_workflow records with stage=detected, updates API status to zombie	P0
FR4.5	User can dismiss individual findings as false positives	P1
FR4.6	Audit trail entries created for all import/decommission actions	P1

FR5: Dashboard & Reporting

ID	Requirement	Priority
FR5.1	Scanner overview page shows: total targets, recent jobs, classification summary chart	P0
FR5.2	Scan job detail page shows: real-time progress bar, results breakdown (Recharts donut), endpoint table	P0
FR5.3	Zombie Graveyard page updated to show scanner-detected zombies with scan source	P1

FR6: Mock Target Website

ID	Requirement	Priority
FR6.1	Next.js app with login page (mock OAuth — click Sign in, stores JWT in localStorage)	P0
FR6.2	Pages: Dashboard, Accounts, Payments, Transactions, User Profile	P0
FR6.3	Each page calls mock API endpoints on load + user interaction	P0
FR6.4	Hidden pages: /admin (user listing), /debug (env/config)	P0
FR6.5	Mock Express API server (within same container or separate)	P0
FR6.6	APIs that are documented (exist in ZADDP seed data) — e.g. GET /api/v1/accounts/{id}/balance	P0
FR6.7	APIs that are shadow (not in ZADDP seed) — e.g. POST /api/v1/internal/cache/flush, GET /api/v1/admin/users/export	P0
FR6.8	APIs that are zombie (in ZADDP seed as active but not called by any page) — e.g. POST /api/v1/currency/convert, GET /api/v1/investments/portfolio	P0
FR6.9	APIs that are deprecated but still active (in seed as deprecated but still respond) — e.g. GET /legacy/accounts/{id}	P0
FR6.10	Dockerized, runs on port 4000	P0

5. Technical Design

5.1 New Database Models

Three new tables are added to the ZADDP PostgreSQL schema:

scan_targets

Column	Type	Description
id	UUID, PK	Primary key
name	VARCHAR(200)	Human-readable target name
target_url	VARCHAR(500)	Base URL of the target website
auth_type	VARCHAR(20)	oauth2 or cookie
oauth_client_id	VARCHAR(500)	OAuth client ID (nullable)
oauth_client_secret	VARCHAR(500)	OAuth client secret (encrypted)
oauth_token_url	VARCHAR(500)	OAuth token endpoint
oauth_scopes	VARCHAR(500)	Space-separated OAuth scopes
cookie_value	TEXT	Cookie string (for cookie auth)
max_crawl_depth	INTEGER	Max BFS crawl depth (default: 3)
created_at	DATETIME	Creation timestamp

scan_jobs

Column	Type	Description
id	UUID, PK	Primary key
target_id	UUID, FK	FK to scan_targets.id
status	VARCHAR(20)	pending/running/completed/failed
progress_pct	INTEGER	0-100 progress
pages_crawled	INTEGER	Number of pages crawled
total_requests	INTEGER	Total requests intercepted
apis_discovered	INTEGER	Unique endpoints found
apis_matched	INTEGER	Matched to inventory
apis_shadow	INTEGER	Shadow APIs discovered
apis_zombie	INTEGER	Zombie APIs flagged
error_log	TEXT	Error details if failed

scan_results

Column	Type	Description
id	UUID, PK	Primary key
scan_job_id	UUID, FK	FK to scan_jobs.id
api_id	UUID, FK (nullable)	FK to apis.id (null if shadow)
http_method	VARCHAR(10)	GET, POST, PUT, DELETE, etc.
endpoint_path	VARCHAR(500)	Raw discovered URL path

normalized_path	VARCHAR(500)	Normalized path with {param}
classification	VARCHAR(20)	matched/shadow/zombie/deprecated
status_code	INTEGER	HTTP response status code
page_url	VARCHAR(500)	Page where request was found
dismissed	BOOLEAN	Whether user dismissed this finding

5.2 Scanner API Routes

Prefix: /api/v1/scanner

Method	Path	Handler
GET	/targets	list_targets
POST	/targets	create_target
GET	/targets/{id}	get_target
PATCH	/targets/{id}	update_target
DELETE	/targets/{id}	delete_target
POST	/targets/{id}/scan	trigger_scan — enqueues job to Redis
GET	/jobs	list_jobs
GET	/jobs/{id}	get_job
GET	/jobs/{id}/results	get_job_results
POST	/jobs/{id}/import	import_shadow_apis
POST	/jobs/{id}/decommission	decommission_zombie_apis

5.3 Scanning Algorithm

- 1. OAuth Authentication:** POST to token_url with client_id, client_secret, grant_type → receive access_token.
- 2. Launch Playwright:** Start headless Chromium browser instance.
- 3. Configure Context:** Create browser context, set Authorization header with access_token.
- 4. Route Interception:** Register page.on("request") and page.on("response") handlers to capture all network traffic.
- 5. BFS Crawl:** Start at target_url, discover <a> links, crawl up to max_crawl_depth. Scroll pages to trigger lazy-loaded API calls.
- 6. Fuzzing Phase:** Try common API paths from a wordlist against the target domain to find undocumented endpoints.
- 7. Normalize Paths:** Replace UUIDs with {uuid}, numeric IDs with {id}, hashes with {hash}.
- 8. Deduplicate:** Group by (method, normalized_path), keep the first occurrence.
- 9. Classify:** Compare each discovered endpoint against the apis table. Label as matched, shadow, zombie, or deprecated.
- 10. Persist:** Bulk-insert scan_results, update scan_job with summary counts, set status to completed.

5.4 Frontend Pages

Route	Page	Key Components
/scanner	Scanner Dashboard	Target list, recent jobs summary, classification donut chart
/scanner/targets/new	New Target	ScanTargetForm (URL, OAuth fields, auth type selector)
/scanner/targets/[id]	Target Detail	Target info, scan history table, Trigger Scan button
/scanner/jobs/[id]	Job Detail	Progress bar, results table, classification breakdown, Import/Decommission actions

5.5 Mock Target Website Structure

```
mock-target/
  ■■ package.json
  ■■ next.config.js
  ■■ Dockerfile
  ■■ app/
    ■■ layout.tsx
    ■■ login/page.tsx (mock OAuth login)
    ■■ dashboard/page.tsx (calls stats APIs)
    ■■ accounts/page.tsx
    ■■ accounts/[id]/page.tsx
    ■■ payments/page.tsx
    ■■ admin/page.tsx (hidden – shadow APIs)
    ■■ debug/page.tsx (hidden – shadow APIs)
  ■■ api/server.ts (Express mock API backend)
```

API Categories:

Documented: GET /api/v1/accounts, /accounts/{id}, /accounts/{id}/balance, /transactions, /payments/history — all exist in ZADDP seed.

Shadow: POST /api/v1/internal/cache/flush, GET /api/v1/admin/users/export, POST /api/v1/experimental/ai/score — NOT in ZADDP seed.

Zombie: POST /api/v1/currency/convert, GET /api/v1/investments/portfolio — in seed as active but never called by any page.

Deprecated: GET /legacy/accounts/{id}, GET /legacy/transactions/{id} — in seed as deprecated but still respond.

6. Non-Functional Requirements

ID	Requirement	Target
NFR1	Scan completion time for mock target	<5 minutes
NFR2	Maximum simultaneous scans	2 (configurable)
NFR3	API response time for scanner CRUD endpoints	<200ms
NFR4	Playwright runs in headless mode only	Never launches visible browser
NFR5	OAuth secrets encrypted at rest	Via application-level encryption or vault
NFR6	Scanner worker auto-restarts on crash	Docker restart policy

7. Implementation Phases & Effort

Phase	Description	Files	Est. Effort
P1	DB models + enums + schemas	models.py, schemas.py	1 session
P2	Scanner API routes (CRUD targets, trigger scan)	routers/scanner.py	1 session
P3	Scanner worker + Playwright service	scanner_worker.py, services/scanner.py, Dockerfile.scanner	2 sessions
P4	Redis queue integration (producer in API, consumer in worker)	main.py, scanner_worker.py	1 session
P5	Frontend: scanner pages + components	5 pages, 6 components, api.ts, Sidebar.tsx	2 sessions
P6	Mock target website	Full mock-target/ directory	2 sessions
P7	Docker Compose + Nginx updates	docker-compose.yml, nginx.conf	1 session
P8	Integration testing (scan mock target, verify in ZADDP)	—	1 session

8. Edge Cases

Edge Case	Handling
Target website requires MFA	Scanner marks job as failed, notes MFA detected in error_log
OAuth token expires mid-scan	Scanner refreshes token (if refresh_token available) or fails gracefully
No APIs discovered (non-SPA target)	Scanner reports 0 discovered but notes target URL was reachable

Target has infinite scroll	Scanner scrolls to bottom x configurable scroll limit
Target behind corporate SSO requiring device auth	Scanner cannot complete — mark as unsupported auth type
Shadow API is actually a 3rd-party/CDN URL	URL filter only captures same-domain or API-subdomain requests
Zombie API still receives traffic from non-browser clients	Classification caveat noted in UI: not discovered via crawl — may still have non-browser consumers

9. Open Questions

Question	Recommendation
Should we capture same-origin only, or also subdomains like api.target.com?	Same-origin + configurable allowlist.
Should the scanner throttle requests to avoid triggering WAF/IDS?	Configurable delay between navigations (default 500ms).
Should scan results persist indefinitely or have a retention policy?	Results retained for 90 days, then archived.
Should targets be scoped per user/team?	Global for now (single-tenant demo), add RBAC later.